

CO2 - AT1 - Database Design and Connectivity

NAME:GUNA P

REG NO:192424277

COURSE NAME:DSA0503 - Query Processing for Data Science

1. School Attendance Management System

Aim:

To create Student and Attendance tables in SQLite, insert attendance records, and display attendance details for a specified date.

Algorithm:

1. Import the sqlite3 library.
2. Create a connection to the SQLite database.
3. Create the **Student** table.
4. Create the **Attendance** table with a foreign key referencing the Student table.
5. Insert sample student records.
6. Insert attendance records.
7. Ask the user to enter a date.
8. Join the Student and Attendance tables.
9. Display attendance details for the entered date.
10. Close the database connection.

CODE:

```
import sqlite3
```

```
conn = sqlite3.connect("school.db")
```

```
cursor = conn.cursor()
```

```
cursor.execute("""
```

```
CREATE TABLE IF NOT EXISTS Student(
```

```
    Student_ID INTEGER PRIMARY KEY,
```

```

        Name TEXT NOT NULL,

        Class TEXT NOT NULL

    )

    """)

cursor.execute("""

CREATE TABLE IF NOT EXISTS Attendance(

    Attendance_ID INTEGER PRIMARY KEY,

    Student_ID INTEGER,

    Date TEXT,

    Status TEXT,

    FOREIGN KEY(Student_ID) REFERENCES Student(Student_ID)

)

    """)

cursor.execute("DELETE FROM Attendance")

cursor.execute("DELETE FROM Student")

students = [

    (1, "Arun", "CSE-A"),

    (2, "Priya", "CSE-A"),

    (3, "Rahul", "CSE-B")

]

cursor.executemany("INSERT INTO Student VALUES(?,?,?)", students)

attendance = [

    (1, 1, "2026-07-28", "Present"),

```

```

(2, 2, "2026-07-28", "Absent"),
(3, 3, "2026-07-28", "Present"),
(4, 1, "2026-07-29", "Present"),
(5, 2, "2026-07-29", "Present"),
(6, 3, "2026-07-29", "Absent")
]

cursor.executemany("INSERT INTO Attendance VALUES(?,?,?,?)",
attendance)

conn.commit()

date = input("Enter Date (YYYY-MM-DD): ")

cursor.execute("""
SELECT Student.Student_ID,
        Student.Name,
        Student.Class,
        Attendance.Date,
        Attendance.Status
FROM Student
JOIN Attendance
ON Student.Student_ID = Attendance.Student_ID
WHERE Attendance.Date = ?
""", (date,))

rows = cursor.fetchall()

print("\nAttendance Details")

print("-"*60)

```

for row in rows:

```
print("Student ID :", row[0])
```

```
print("Name      :", row[1])
```

```
print("Class     :", row[2])
```

```
print("Date      :", row[3])
```

```
print("Status    :", row[4])
```

```
print("-"*60)
```

```
conn.close()
```

OUTPUT:

```
Enter Date (YYYY-MM-DD): 2026-07-28

Attendance Details
-----
Student ID : 1
Name       : Arun
Class      : CSE-A
Date       : 2026-07-28
Status     : Present
-----
Student ID : 2
Name       : Priya
Class      : CSE-A
Date       : 2026-07-28
Status     : Absent
-----
Student ID : 3
Name       : Rahul
Class      : CSE-B
Date       : 2026-07-28
Status     : Present
-----
```

2. Banking Customer Database:

Aim:

To design a Banking Customer Database using MySQL by creating an **Account** table, inserting customer account records, updating the account balance after a transaction, and retrieving the updated account information.

Algorithm:

1. Install and start the MySQL (MariaDB) server.
2. Install the MySQL connector library.
3. Create the Bank database.
4. Connect Python to the MySQL database.
5. Create the Account table with appropriate constraints.
6. Insert customer account records.
7. Ask the user for an account number and deposit amount.
8. Update the account balance.
9. Retrieve and display the updated account details.
10. Close the database connection.

CODE:

```
import sqlite3

import pandas as pd

conn = sqlite3.connect("BankDB.db")

cursor = conn.cursor()

cursor.execute("""

CREATE TABLE IF NOT EXISTS Account(

    Account_No INTEGER PRIMARY KEY,

    Customer_Name TEXT NOT NULL,

    Account_Type TEXT NOT NULL,
```

```

        Balance REAL NOT NULL CHECK(Balance>=0)
    )
    """)
cursor.execute("DELETE FROM Account")
accounts = [
    (1001, "Arun", "Savings", 25000),
    (1002, "Priya", "Current", 42000),
    (1003, "Rahul", "Savings", 30000),
    (1004, "Sneha", "Savings", 50000)
]
cursor.executemany("INSERT INTO Account VALUES (?,?,,?)",
accounts)
conn.commit()
print("Customer Account Records\n")
df = pd.read_sql_query("SELECT * FROM Account", conn)
print(df)

acc = int(input("\nEnter Account Number: "))
deposit = float(input("Enter Deposit Amount: "))
cursor.execute(
    "UPDATE Account SET Balance = Balance + ? WHERE Account_No
= ?",
    (deposit, acc)
)

```

```

conn.commit()

print("\nUpdated Account Details\n")

df = pd.read_sql_query(
    "SELECT * FROM Account WHERE Account_No=?",
    conn,
    params=(acc,)
)

print(df)

conn.close()

```

OUTPUT:

```

** Customer Account Records

```

	Account_No	Customer_Name	Account_Type	Balance
0	1001	Arun	Savings	25000.0
1	1002	Priya	Current	42000.0
2	1003	Rahul	Savings	30000.0
3	1004	Sneha	Savings	50000.0

```

Enter Account Number: 1002
Enter Deposit Amount: 5000

Updated Account Details

```

	Account_No	Customer_Name	Account_Type	Balance
0	1002	Priya	Current	47000.0